

SOCIAL MEDIA MUTUAL FRIENDS ANALYZER

A Project Report

In the partial fulfillment of the award of the degree of

B.E.

under

Academy of Skill Development



Submitted by

Vishal Shivraj Kumbhar



**Dr.D.Y.Patil College of Engineering and
Innovation,Varale,Talegaon**



Certificate from the Mentor

This is to certify that **Vishal Shivraj Kumbhar** has completed the project titled **Social Media "Mutual Friends" Analyzer** under my supervision during the period from **[Start Month]** to **[End Month]** which is in partial fulfillment of requirements for the award of the **B.E/MCA/BCA** and submitted to Department **Computer Engineering** of **Dr.D.Y.Patil College of Engineering and Innovation** **Varale,Talegaon**

Signature of the Mentor

Date:

Acknowledgment

I take this opportunity to express my deep gratitude and sincerest thanks to my project mentor, **Mr.Dhruba Rai** for giving the most valuable suggestions, helpful guidance, and encouragement in the execution of this project work.

I would like to give a special mention to my colleagues. Last but not least I am grateful to all the faculty members of the **Academy of Skill Development** for their support.

1. Introduction

The **Social Media Mutual Friend Analyzer** is a Java-based application designed to simulate a simplified social networking system. It allows users to create profiles, establish friendships, and analyze relationships between users. The system focuses on identifying **mutual friends** and **suggesting new connections**, similar to features found in real-world platforms like Facebook or LinkedIn.

This project integrates:

- **Core Java programming**
- **Data structures**
- **Networking (HTTP server)**
- **Multithreading**
- **File handling for persistence**

It provides two interfaces:

1. **Command Line Interface (CLI)**
2. **Web-based GUI (via browser)**

2. Objectives

Functional Objectives

The primary goal of the Mutual Friend Analyzer system is to simulate a simplified social network environment where users can manage friendships and analyze connections efficiently. The following functional objectives define what the system is expected to perform:

- **Add Users to the System**

The application allows the creation of multiple users, each identified uniquely (typically using a name or ID). This forms the base structure of the social network, enabling further operations.

- **Add Friends to Users**

Each user can have a list of friends. The system provides functionality to add friends to a specific user's friend list. It ensures that friendships are stored properly and efficiently for further analysis.

- **Find Mutual Friends Between Two Users**

One of the core features of this project is identifying mutual friends between any two users. This is achieved using set operations, specifically intersection logic, to find common elements in both users' friend lists.

- **Display All Users and Their Friend Lists**

The system allows viewing all users along with their respective friend lists. This helps in understanding the overall network structure and verifying stored data.

Technical Objectives

The project is designed not just as an application but also as a practical implementation of important Java concepts, especially data structures. The technical objectives include:

- **Use of Set (HashSet / TreeSet) Effectively**

The system uses Java's Set interface to store friend lists.

- HashSet ensures fast operations like insertion, deletion, and search.
- TreeSet maintains elements in sorted order automatically. This demonstrates efficient data handling and selection of appropriate data structures.

- **Application of Set Algebra Operations**

The project applies mathematical set operations such as:

- **Intersection** → to find mutual friends
 - **Union** → to combine friend lists (if needed for extensions)
 - **Difference** → to find unique friends
- This shows how theoretical concepts are applied in real-world programming.

- **Avoidance of Duplicate Entries**

One major advantage of using sets is that they automatically prevent duplicate values. This ensures that a user's friend list contains only unique entries, maintaining data integrity without extra validation logic.

- **Automatic Sorting Without Manual Effort**

By using TreeSet, the system automatically sorts friend names in ascending order. This removes the need for additional sorting algorithms and simplifies implementation while improving readability of output.

3. System Overview

The Mutual Friend Analyzer system is designed to represent a simplified social network where users and their friendships are efficiently managed using Java data structures. The system focuses on storing, processing, and analyzing relationships between users using a combination of Map and Set collections.

◇ System Architecture

At its core, the system maintains a **mapping between users and their respective friend lists**. This is achieved using a Map data structure, where:

- **Key** → Represents a unique user (e.g., username)
- **Value** → Represents a Set of friends associated with that user

This structure ensures fast access to any user's friend list and supports efficient operations like insertion, retrieval, and updates.

◇ Working of the System

The system operates based on the following logical flow:

1. User Data Storage using Map

All users are stored in a Map (such as HashMap or TreeMap). Each user is uniquely identified and mapped to their corresponding friend list.

Example:

UserA → {Friend1, Friend2, Friend3}

UserB → {Friend2, Friend3, Friend4}

This mapping allows quick lookup of any user and their friends, making the system scalable and efficient.

2. Friend Management using Set

Each user's friend list is stored using a Set implementation:

- HashSet → Provides fast performance for adding and searching friends
- TreeSet → Maintains friends in sorted (alphabetical) order

Using a Set ensures:

- No duplicate friends are stored
 - Efficient operations like add, remove, and search
 - Cleaner and more reliable data handling
-

3. Finding Mutual Friends using Set Intersection

The system identifies mutual friends between two users using **set intersection**.

This is done by comparing their friend lists and extracting common elements.

Logic:

- Retrieve both users' friend sets
- Apply intersection operation (retainAll() in Java)
- The resulting set contains only common friends

Example:

UserA → {A, B, C}

UserB → {B, C, D}

Mutual Friends → {B, C}

This operation is efficient and directly leverages the properties of sets.

4. Sorted Output using TreeSet

To improve readability and user experience, the system displays results in a sorted manner. This is achieved by using TreeSet, which automatically arranges elements in ascending order.

Benefits:

- No need for manual sorting algorithms
- Consistent and clean output format
- Better visualization of data

◇ Key Features of the System

- Efficient storage using Map and Set
- Fast retrieval and update operations
- Automatic handling of duplicate entries
- Mathematical approach to finding mutual connections
- Sorted and user-friendly output

4. Data Structure Used

◇ Main Structure

`Map<String, Set<String>> userFriendsMap;`

🔍 Explanation

- Map → Stores users
 - String (Key) → Username
 - Set<String> (Value) → Friends list
-

Example:

alice → [bob, charlie, david]

john → [charlie, emma, bob]

◇ Types of Set Used

1. HashSet

- Stores friends
- Fast operations
- No duplicates

2. TreeSet

- Used for sorted output
- Maintains alphabetical order

5. Core Concept: Set Algebra

Set Algebra is a mathematical concept applied to programming.

◇ 1. Intersection (Mutual Friends)

Definition:

Common elements between two sets

Java Method:

`retainAll()`

Example:

A = [bob, charlie, david]

B = [charlie, david, emma]

$A \cap B = [\text{charlie, david}]$

◇ 2. Union

Definition:

All elements from both sets

`addAll()`

◇ 3. Difference

Definition:

Elements in one set but not in another

`removeAll()`

6. Algorithm for Mutual Friends

Steps:

1. Retrieve friend list of User A
 2. Retrieve friend list of User B
 3. Convert one list into **TreeSet**
 4. Apply retainAll()
 5. Return result
-

Code Implementation:

```
public static Set<String> getMutualFriends(String user1, String user2,
                                           Map<String, Set<String>> map) {

    if (!map.containsKey(user1) || !map.containsKey(user2)) {
        return null;
    }

    Set<String> mutual = new TreeSet<>(map.get(user1));
    mutual.retainAll(map.get(user2));

    return mutual;
}
```

7. Key Feature: Automatic Sorting

Why TreeSet?

- Maintains **natural ordering** (alphabetical)
 - No need for manual sorting
 - Cleaner and efficient code
-

Without TreeSet:

- Need extra sorting logic ✕

With TreeSet:

- Sorted automatically ☑
-

9. Modules Description

◇ 1. Main Module

- Controls program flow
 - Takes user input
 - Calls required functions
-

◇ 2. User Management

- Add user
 - Store user in map
-

◇ 3. Friend Management

- Add friends
 - Store in Set
-

◇ 4. Mutual Friend Analyzer

- Uses Set intersection
 - Returns sorted result
-

10. Advantages

- No duplicate entries
 - Fast operations
 - Simple implementation
 - Uses built-in Java methods
 - Automatic sorting with TreeSet
-

11. Limitations

- Works on local data only
- No database integration
- Basic interface
- Limited scalability

12. Future Enhancements

- Add database (MySQL)
- Build web interface
- Add friend suggestions
- Visualize network graph
- Deploy as web application

13. Conclusion

The project successfully demonstrates the use of **Set and Set Algebra** in solving real-world problems like mutual friend detection in social networks.

By using:

- **HashSet** → for efficient storage
- **TreeSet** → for sorted output
- **retainAll()** → for intersection

the system achieves:

- Clean logic
- High efficiency
- Readable implementation

This project provides a strong foundation for understanding **data structures and real-world applications**.